```
!pip install rdkit

!pip install torch-geometric
```

Gizli çıkışı göster

```
import sys, subprocess, pkgutil

def ensure(pkg, install_name=None):
    if pkgutil.find_loader(pkg) is None:
        subprocess.check_call([sys.executable, "-m", "pip", "install", install_name or pkg])

ensure("rdkit", "rdkit-pypi")
ensure("torch_geometric", "torch-geometric")

import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
from rdkit import Chem
from rdkit.Chem import rdmolops, HybridizationType

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

from torch_geometric.data import Data
from torch_geometric.loader import DataLoader
from torch_geometric.nn import GATConv, global_mean_pool

print("Torch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

```
/tmp/ipykernel_571/1843802349.py:5: DeprecationWarning: 'pkgutil.find_loader' is deprecated and slated for remova
  if pkgutil.find_loader(pkg) is None:
Torch: 2.10.0+cpu
CUDA available: False
```

```
SEED = 42
BATCH_SIZE = 32
LR = 3e-4
WEIGHT_DECAY = 1e-4
NUM_EPOCHS = 120
PATIENCE = 12

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)

set_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def atom_features(atom):
    return [
        atom.GetAtomicNum(),
        atom.GetTotalDegree(),
        atom.GetFormalCharge(),
        int(atom.GetHybridization() == HybridizationType.SP),
        int(atom.GetHybridization() == HybridizationType.SP2),
        int(atom.GetHybridization() == HybridizationType.SP3),
        int(atom.GetIsAromatic()),
        int(atom.IsInRing())
    ]

def mol_to_graph(smiles, label):
    mol = Chem.MolFromSmiles(smiles)
    if mol is None or mol.GetNumAtoms() == 0:
        return None
    adj = rdmolops.GetAdjacencyMatrix(mol)
    feats = [atom_features(atom) for atom in mol.GetAtoms()]
    x = torch.tensor(feats, dtype=torch.float)
    edge_index = torch.tensor(np.array(np.nonzero(adj)), dtype=torch.long)
    y = torch.tensor([float(label)], dtype=torch.float)
```

```
    return Data(x=x, edge_index=edge_index, y=y)

df = pd.read_csv("https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv")
df = df.dropna(subset=["smiles", "HIV_active"]).reset_index(drop=True)

graphs = []
labels = []
for _, row in df.iterrows():
    g = mol_to_graph(row["smiles"], row["HIV_active"])
    if g is not None:
        graphs.append(g)
        labels.append(int(row["HIV_active"]))

labels = np.array(labels)
print("Total valid graphs:", len(graphs))
print("Positive ratio:", labels.mean())
```

```
Total valid graphs: 2885
Positive ratio: 0.5105719237435009
```

```
class GATModel(nn.Module):
    def __init__(self, in_channels=8):
        super().__init__()
        self.gat1 = GATConv(in_channels, 128, heads=4, concat=False, dropout=0.3)
        self.bn1  = nn.BatchNorm1d(128)
        self.gat2 = GATConv(128, 64, heads=4, concat=False, dropout=0.3)
        self.bn2  = nn.BatchNorm1d(64)
        self.dropout = nn.Dropout(0.4)
        self.lin1 = nn.Linear(64, 32)
        self.lin2 = nn.Linear(32, 1)

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch
        x = F.elu(self.gat1(x, edge_index))
        x = self.bn1(x)
        x = F.elu(self.gat2(x, edge_index))
        x = self.bn2(x)
        x = global_mean_pool(x, batch)
        x = self.dropout(x)
        x = F.relu(self.lin1(x))
        return torch.sigmoid(self.lin2(x)).view(-1)

def evaluate_model(model, loader, loss_fn):
    model.eval()
    total_loss = 0.0
    preds, labels = [], []
    with torch.no_grad():
        for data in loader:
            data = data.to(device)
            out = model(data)
            loss = loss_fn(out, data.y.view(-1).float())
            total_loss += loss.item() * data.num_graphs
            preds.extend(out.detach().cpu().numpy().tolist())
            labels.extend(data.y.view(-1).cpu().numpy().tolist())
    avg_loss = total_loss / len(loader.dataset)
    pred_binary = [1 if p >= 0.5 else 0 for p in preds]
    acc = accuracy_score(labels, pred_binary)
    precision = precision_score(labels, pred_binary, zero_division=0)
    recall = recall_score(labels, pred_binary, zero_division=0)
    f1 = f1_score(labels, pred_binary, zero_division=0)
    auc = roc_auc_score(labels, preds) if len(set(labels)) > 1 else 0.0
    return avg_loss, acc, precision, recall, f1, auc, preds, labels

def train_one_fold(train_graphs, val_graphs, test_graphs, fold_id):
    train_loader = DataLoader(train_graphs, batch_size=BATCH_SIZE, shuffle=True)
    val_loader   = DataLoader(val_graphs, batch_size=BATCH_SIZE, shuffle=False)
    test_loader  = DataLoader(test_graphs, batch_size=BATCH_SIZE, shuffle=False)

    model = GATModel().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
    loss_fn = nn.BCELoss()
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode="min", factor=0.5, patience=5
    )

    best_val_loss = float("inf")
    best_state_dict = None
    wait = 0

    train_losses, val_losses = [], []
    train_accs, val_accs = [], []
    train_aucs, val_aucs = [], []

    for epoch in range(1, NUM_EPOCHS + 1):
        model.train()
        total_loss = 0.0
        train_preds, train_labels = [], []

        for data in train_loader:
```

```
            data = data.to(device)
            optimizer.zero_grad()
            out = model(data)
            loss = loss_fn(out, data.y.view(-1).float())
            loss.backward()
            optimizer.step()

            total_loss += loss.item() * data.num_graphs
            train_preds.extend(out.detach().cpu().numpy().tolist())
            train_labels.extend(data.y.view(-1).cpu().numpy().tolist())

        avg_train_loss = total_loss / len(train_loader.dataset)
        train_pred_binary = [1 if p >= 0.5 else 0 for p in train_preds]
        train_acc = accuracy_score(train_labels, train_pred_binary)
        train_auc = roc_auc_score(train_labels, train_preds) if len(set(train_labels)) > 1 else 0.0

        val_loss, val_acc, _, _, _, val_auc, _, _ = evaluate_model(model, val_loader, loss_fn)
        scheduler.step(val_loss)

        train_losses.append(avg_train_loss)
        val_losses.append(val_loss)
        train_accs.append(train_acc)
        val_accs.append(val_acc)
        train_aucs.append(train_auc)
        val_aucs.append(val_auc)

        if val_loss < best_val_loss - 1e-4:
            best_val_loss = val_loss
            best_state_dict = {k: v.cpu().clone() for k, v in model.state_dict().items()}
            wait = 0
        else:
            wait += 1
            if wait >= PATIENCE:
                print(f"Fold {fold_id}: Early stopping at epoch {epoch}")
                break

    if best_state_dict is not None:
        model.load_state_dict(best_state_dict)

    test_loss, test_acc, test_precision, test_recall, test_f1, test_auc, _, _ = evaluate_model(model, test_loader, loss_fn)
    history = {
        "train_losses": train_losses,
        "val_losses": val_losses,
        "train_accs": train_accs,
        "val_accs": val_accs,
        "train_aucs": train_aucs,
        "val_aucs": val_aucs,
    }
    metrics = {
        "fold": fold_id,
        "test_accuracy": test_acc,
        "test_precision": test_precision,
        "test_recall": test_recall,
        "test_f1": test_f1,
        "test_roc_auc": test_auc,
        "epochs_ran": len(train_losses),
    }
    return metrics, history
```

```
all_fold_results = []
all_histories = []

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
indices = np.arange(len(graphs))

for fold_id, (train_idx, temp_idx) in enumerate(skf.split(indices, labels), start=1):
    train_idx = np.array(train_idx)
    temp_idx = np.array(temp_idx)

    temp_labels = labels[temp_idx]
    val_sub_idx, test_sub_idx = train_test_split(
        np.arange(len(temp_idx)),
        test_size=0.5,
        random_state=SEED + fold_id,
        stratify=temp_labels
    )

    train_graphs = [graphs[i] for i in train_idx]
    val_graphs = [graphs[temp_idx[i]] for i in val_sub_idx]
    test_graphs = [graphs[temp_idx[i]] for i in test_sub_idx]

    print(f"===== Fold {fold_id} =====")
    print("Train:", len(train_graphs), "Val:", len(val_graphs), "Test:", len(test_graphs))

    fold_result, history = train_one_fold(train_graphs, val_graphs, test_graphs, fold_id)
    print(fold_result)

    all_fold_results.append(fold_result)
    all_histories.append(history)
```

```
results_df = pd.DataFrame(all_fold_results)
results_df
```

```
===== Fold 1 =====
Train: 2308 Val: 288 Test: 289
Fold 1: Early stopping at epoch 54
{'fold': 1, 'test_accuracy': 0.6539792387543253, 'test_precision': 0.6620689655172414, 'test_recall': 0.653061224
===== Fold 2 =====
Train: 2308 Val: 288 Test: 289
Fold 2: Early stopping at epoch 58
{'fold': 2, 'test_accuracy': 0.698961937716263, 'test_precision': 0.7205882352941176, 'test_recall': 0.6666666666
===== Fold 3 =====
Train: 2308 Val: 288 Test: 289
Fold 3: Early stopping at epoch 83
{'fold': 3, 'test_accuracy': 0.6643598615916955, 'test_precision': 0.650887573964497, 'test_recall': 0.7432432432
===== Fold 4 =====
Train: 2308 Val: 288 Test: 289
Fold 4: Early stopping at epoch 46
{'fold': 4, 'test_accuracy': 0.6851211072664359, 'test_precision': 0.7244094488188977, 'test_recall': 0.621621621
===== Fold 5 =====
Train: 2308 Val: 288 Test: 289
Fold 5: Early stopping at epoch 79
{'fold': 5, 'test_accuracy': 0.6920415224913494, 'test_precision': 0.6855345911949685, 'test_recall': 0.736486486
```

	fold	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc	epochs_ran
0	1	0.653979	0.662069	0.653061	0.657534	0.730622	54
1	2	0.698962	0.720588	0.666667	0.692580	0.759749	58
2	3	0.664360	0.650888	0.743243	0.694006	0.781052	83
3	4	0.685121	0.724409	0.621622	0.669091	0.782682	46
4	5	0.692042	0.685535	0.736486	0.710098	0.789774	79

```
results_df.to_csv("GAT_fold_results.csv", index=False)

summary_rows = []
for metric in ["test_accuracy", "test_precision", "test_recall", "test_f1", "test_roc_auc"]:
    mean_val = results_df[metric].mean()
    std_val = results_df[metric].std(ddof=1)
    var_val = results_df[metric].var(ddof=1)
    summary_rows.append({
        "Metric": metric.replace("test_", "").upper(),
        "Mean": mean_val,
        "Std": std_val,
        "Variance": var_val,
        "Formatted": f"{mean_val:.3f} ± {std_val:.3f}",
    })

summary_df = pd.DataFrame(summary_rows)
summary_df.to_csv("GAT_summary_results.csv", index=False)
summary_df
```

	Metric	Mean	Std	Variance	Formatted
0	ACCURACY	0.678893	0.019015	0.000362	0.679 ± 0.019
1	PRECISION	0.688698	0.033321	0.001110	0.689 ± 0.033
2	RECALL	0.684216	0.053416	0.002853	0.684 ± 0.053
3	F1	0.684662	0.021067	0.000444	0.685 ± 0.021
4	ROC_AUC	0.768776	0.024095	0.000581	0.769 ± 0.024

```
final_table = pd.DataFrame([{
    "Model": "GAT",
    "Accuracy": summary_df.loc[summary_df["Metric"] == "ACCURACY", "Formatted"].iloc[0],
    "Precision": summary_df.loc[summary_df["Metric"] == "PRECISION", "Formatted"].iloc[0],
    "Recall": summary_df.loc[summary_df["Metric"] == "RECALL", "Formatted"].iloc[0],
    "F1": summary_df.loc[summary_df["Metric"] == "F1", "Formatted"].iloc[0],
    "ROC-AUC": summary_df.loc[summary_df["Metric"] == "ROC_AUC", "Formatted"].iloc[0],
}])

final_table.to_csv("GAT_final_table.csv", index=False)
final_table
```

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	GAT	0.679 ± 0.019	0.689 ± 0.033	0.684 ± 0.053	0.685 ± 0.021	0.769 ± 0.024

```
# Ortalama eğitim grafikleri
def pad_and_average(series_list):
    max_len = max(len(x) for x in series_list)
    arr = np.full((len(series_list), max_len), np.nan, dtype=float)
    for i, s in enumerate(series_list):
        arr[i, :len(s)] = s
    return np.nanmean(arr, axis=0)

avg_train_acc = pad_and_average([h["train_accs"] for h in all_histories])
avg_val_acc   = pad_and_average([h["val_accs"] for h in all_histories])
avg_train_loss = pad_and_average([h["train_losses"] for h in all_histories])
avg_val_loss   = pad_and_average([h["val_losses"] for h in all_histories])

epochs = np.arange(1, len(avg_train_acc) + 1)

plt.figure(figsize=(8,4))
plt.plot(epochs, avg_train_acc, label="Train")
plt.plot(epochs, avg_val_acc, label="Validation")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("GAT Average Accuracy ")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(8,4))
plt.plot(epochs, avg_train_loss, label="Train")
plt.plot(epochs, avg_val_loss, label="Validation")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("GAT Average Loss")
plt.legend()
plt.grid(True)
plt.show()
```



